DevOps Project Report: Scientific Calculator

Name: Abhijeet Rai

Roll Number: MT2025004

---

1. Introduction to DevOps

DevOps is a set of practices that combines software development (Dev) and IT operations (Ops) to improve collaboration and automate the software delivery process. The main goal of devOps is to develop, test, and release software faster and more reliably. It uses practices like Continuous Integration (CI), Continuous Deployment (CD) and automation to reduce errors and improve efficiency in the software development lifecycle.

Traditional software development relies on manual build, testing, and development processes, which can be slow, inconsistent, and prone to human errors. DevOps addresses these issues by automating repetitive tasks, enabling faster feedback, improving code quality, and ensuring consistent application deployment.

.

This project mainly focuses on building **Scientific Calculator** using modern DevOps practices. The application is built using Java and supports the following operations:

- **Square root function -** $\sqrt{x}$
- **Factorial function -** $x!$
- **Natural logarithm (base e) -** $\ln(x)$
- **Power function -** x^b

2. Tools used

Tools	Purpose
Git & GitHub	Source Code Management
C++	Application logic
Make	Unit Testing
Make	Build Automation
Jenkins	Continuous Integration & Delivery (CI/CD)
Docker	Containerization
Docker Hub	Image registry
Ansible	Automated deployment

3. Project Setup

Git Commands

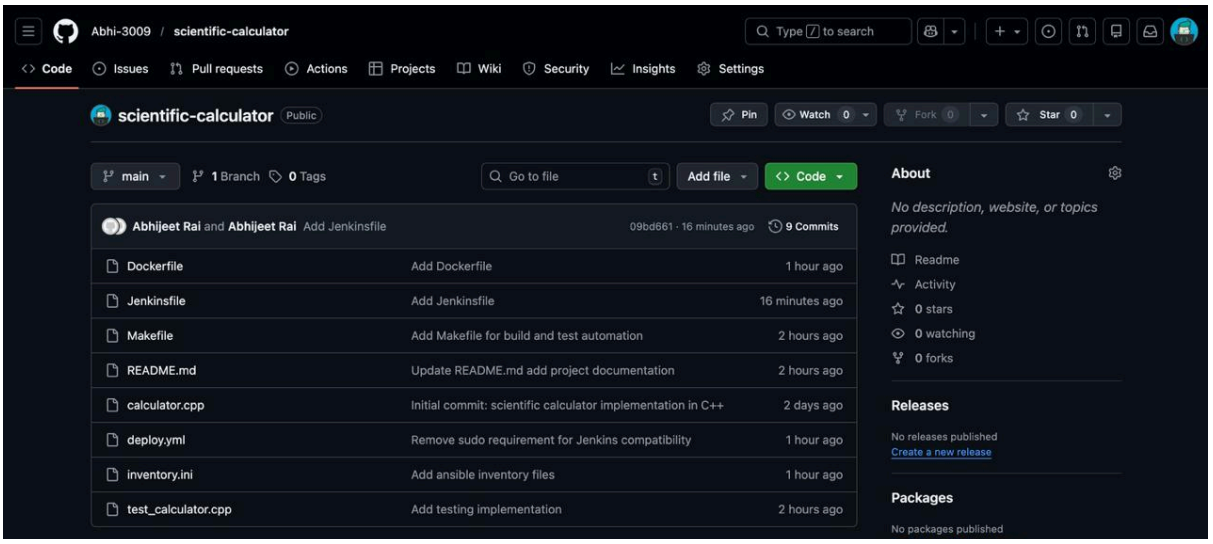
git clone https://github.com/Abhi-3009/scientific-calculator.git

git add .

git commit -m “Changes”

git push origin main

Screenshot 1: GitHub Repository



4. Testing

Test cases:

- Valid edge cases (e.g. factorial of negative number, square root of negative number, etc)
- Parameterized tests for multiple inputs.

Code Snippet: test_calculator.cpp

```cpp
#include <cassert>
#include <cmath>
#include <iostream>
using namespace std;

double squareRoot(double n){
    if (n < 0)
        throw invalid_argument("negative");
    return sqrt(n);
}

long factorial(int n){
    if (n < 0)
        throw invalid_argument("negative");
    long r = 1;
    for (int i = 1; i <= n; i++)
        r *= i;
    return r;
}

double natural_log(double n){
    if (n <= 0)
        throw invalid_argument("invalid");
    return log(n);
}

double power(double b, int e){
    return pow(b, e);
}

```

```cpp
int main()
{
    // square root tests
    assert(squareRoot(9) == 3);
    assert(squareRoot(16) == 4);

    // factorial tests
    assert(factorial(5) == 120);
    assert(factorial(0) == 1);

    // log tests
    assert(abs(natural_log(1)) < 0.0001);

    // power tests
    assert(power(2, 3) == 8);
    assert(power(5, 0) == 1);

    cout << "All tests passed\n";
}
```

5. Build with Make

Makefile configuration:

- Define compilation rules to build the calculator application from calculator.cpp.
- Create build targets for compiling both the main program and the program (test_calculator.cpp).

Make build command:

make

6. Continuous Integration with Jenkins

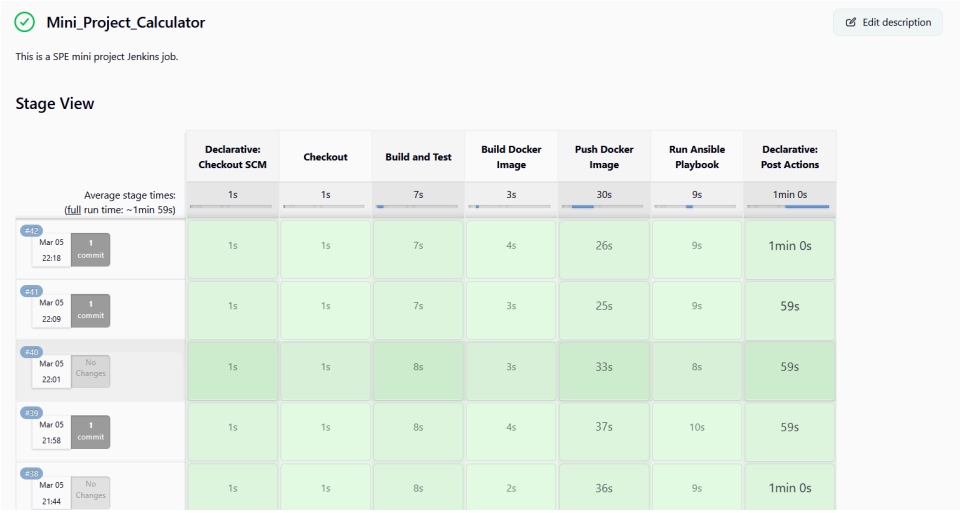
Jenkins Pipeline:

- **Stages:** Checkout → Build → Test → Build Docker Image → Push Docker Image → Ansible Deployment

Code Snippet: Jenkinsfile

```
Jenkinsfile
pipeline {
    agent any
    stages {

        stage('Clone Repository') {
            steps {
                git branch: 'main', url: 'https://github.com/Abhi-3009/scientific-calculator'
            }
        }

        stage('Build') {
            steps {
                sh 'make build'
            }
        }

        stage('Run Tests') {
            steps {
                sh 'make test'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh '/usr/local/bin/docker build -t codestack123/calculator:latest .'
            }
        }

        stage('Push Docker Image') {
            steps {
                sh '/usr/local/bin/docker push codestack123/calculator:latest'
            }
        }

        stage('Deploy with Ansible') {
            steps {
                sh 'ansible-playbook deploy.yml -i inventory.ini'
            }
        }
    }
}
```

Screenshot 4: Full jenkins pipeline view



7. Containerization with Docker

Code Snippet: Dockerfile

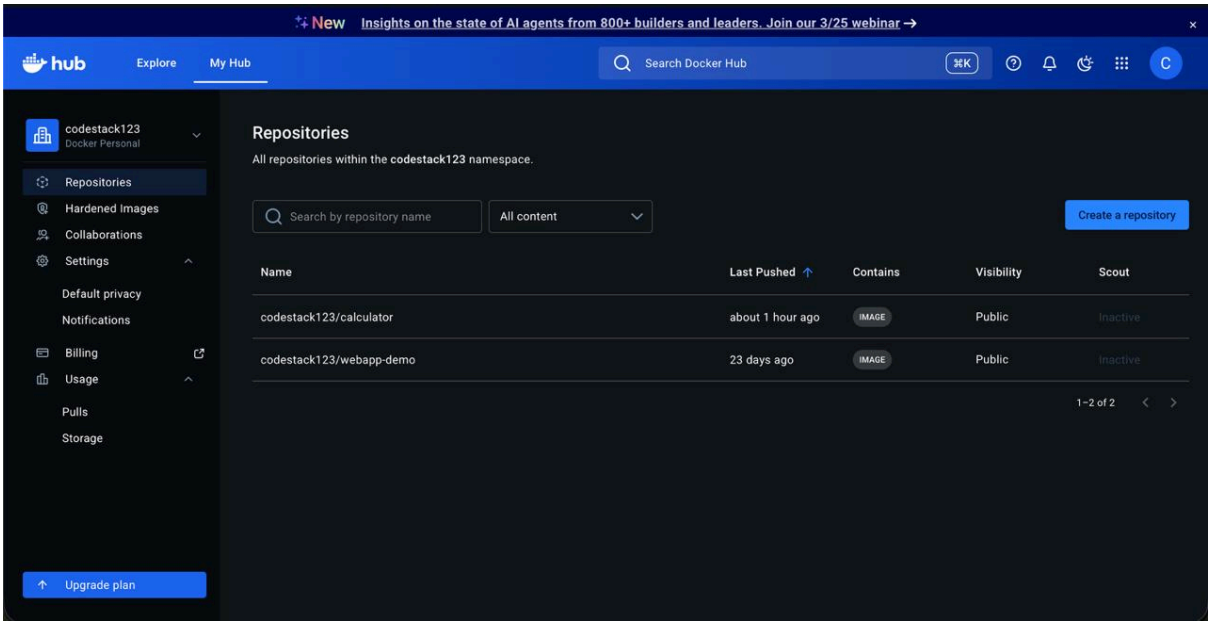
```
FROM gcc:latest

WORKDIR /app

COPY . .

RUN make build

CMD ["./calculator"]
```

Docker commands:

docker build codestack123/scientific-calculator:latest .

docker push codestack123/scientific-calculator:latest

Screenshot 5: Docker image(s) on Docker Hub



8. Deployment with Ansible

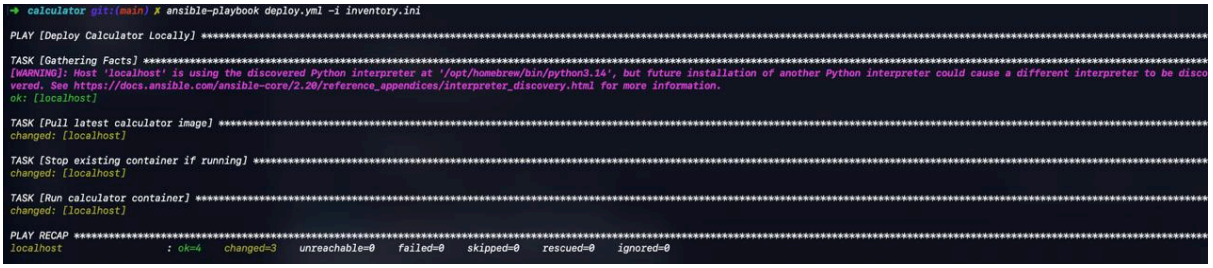
Code Snippet: deploy.yml

```yaml
---
- name: Deploy Calculator Locally
  hosts: local

  tasks:
    - name: Pull latest calculator image
      command: docker pull codestack123/calculator:latest

    - name: Stop existing container if running
      command: docker rm -f calculator
      ignore_errors: yes

    - name: Run calculator container
      command: docker run -it -d --name calculator codestack123/calculator:latest
```

Code Snippet: inventory.ini

```ini
[local]
localhost ansible_connection=local
```

Screenshot 6: Ansible Deployment Output



9. Application Screenshots

Screenshot 7: Application's Command Line Interface (CLI)

```
● → calculator git:(main) ✗ g++ calculator.cpp -o calculator
● → calculator git:(main) ✗ ./calculator
  ------------------------------------
  === Scientific Calculator ===
  ------------------------------------
  Choose an operation:
  1. Square Root
  2. Factorial
  3. Natural Logarithm
  4. Power
  5. Exit
  ------------------------------------
  Enter your choice: 2
  Enter a number: 34

  Result: 4.92628e+18

  ------------------------------------
  === Scientific Calculator ===
  ------------------------------------
  Choose an operation:
  1. Square Root
  2. Factorial
  3. Natural Logarithm
  4. Power
  5. Exit
  ------------------------------------
  Enter your choice: 5

  Exiting Scientific Calculator...
```

10. Conclusion

This project shows how to create and implement a **Scientific Calculator** application using DevOps practices. The project ensures that the application can be built, tested, packaged, and deployed consistently by incorporating automation into various phases of the software development lifecycle. The development process is more efficient due to automated pipeline, which also reduces the possibility of human error and manual interactability.

Links:

- GitHub Repository: https://github.com/Abhi-3009/scientific-calculator
- Docker Repository: https://hub.docker.com/r/codestack123/calculator